

By the end of today, a computer you have never touched will answer to you.

This morning you launch a real server in a real data centre and give it a public address. This afternoon you stop clicking at it and start driving it from a terminal. Everything you deploy for the rest of the week lands on the machine you create today.

IN PARTNERSHIP WITH



Five days, one app, live on the internet

DAY 1 A server of your own Launch it, give it a public address, and drive it from the terminal. ECS EIP SSH	DAY 2 Put it in a container Package the app and deploy it – first by hand, then automatically. Docker GitHub	DAY 3 The network it lives on IPs, subnets and firewalls – why traffic reaches you, or does not. VPC Subnets	DAY 4 Managed services & scale Stop running things yourself, and survive more than one visitor. OSS K8s	DAY 5 Polish and present Document the whole thing and show it to the room. Docs Demo
--	---	---	--	---

Every day adds one piece to the same project. Nothing this week is a throwaway exercise.

One badge page, on the internet, that is unmistakably yours

It is a real web app NODE

A small "Cloud Explorer Badge" page with a photo-booth feature. It runs on your laptop today and on your server by the end of the week.

You make it yours PERSONALISE

Your name, your colours, your favourite movie. Two students' badges should never look the same.

It ends up public DEPLOYED

By Day 5 it answers on a public address that anyone in the room – or anyone at all – can open.

You are not building a demo you throw away on Friday. You are building the thing you will show people.

Get these four installed before we go any further

A GitHub account

DAY 2 ONWARD

Where your project lives and where deployment is triggered from.

Sign up at github.com/signup with an email you will keep.

VS Code

EDITOR

Your editor for the week.

From code.visualstudio.com.

Node.js v20+

RUNTIME

So you can run the lab project on your own laptop before it ever reaches a server.

From nodejs.org – check with `node -v`.

Docker Desktop

DAY

2

You build and push your own image with it tomorrow.

From docker.com. On macOS, follow the [setup guide](#) by [Lama Alothman](#).

This costs real money and is really on the internet. One small server for the whole week. Do not spin up extras, and follow the cleanup steps on Friday.

Saudi AZM builds the systems other Saudi organisations run on



A leading IT company in Riyadh. Both private- and government-sector entities rely on AZM to build their digital solutions.

AZEM Technical عازم

A fully remote graduate programme across Cloud Computing, Software Testing & QA and Software Development. Six months that pack in about a year of content. Currently in its 2nd batch; the next starts around the end of 2026.

What a trainee actually touches

- Backend: PHP Laravel · Node · Python · Golang
- Messaging: RabbitMQ · MQTT
- Search: Elasticsearch · OpenSearch
- Containers: Kubernetes · OpenShift
- Observability: ELK · Prometheus / Grafana

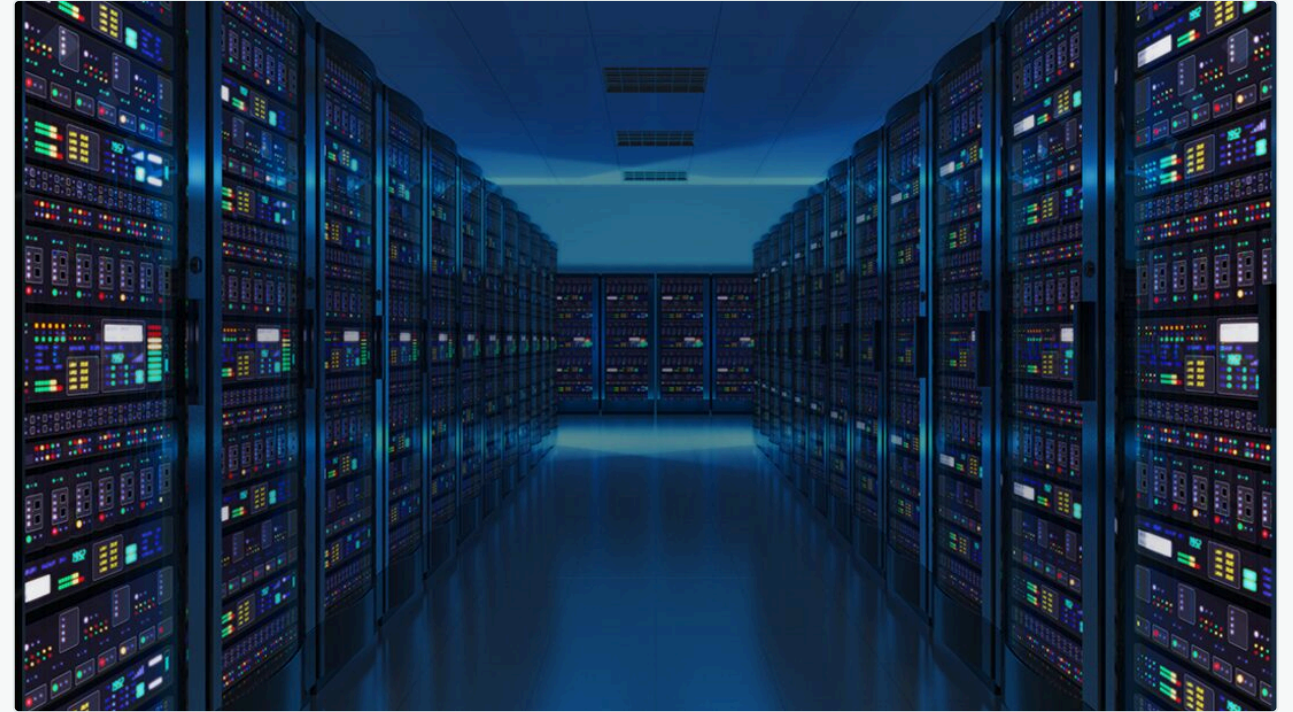
Real projects with real impact – other teams depend on your work.

Questions about the programme: ask your instructor Ali, or ali.aljaffer@azm.com.

The cloud is renting someone else's computers, by the hour

That is genuinely it. The cloud is not magic and it is not one place – it is giant buildings full of computers owned by a provider, and you rent slices of them through a website called the console.

You can give the slice back the moment you are done, and stop paying for it. This week your provider is Alibaba Cloud.



A data centre – the building your "cloud" server is physically standing in.

Why anyone would rent instead of just buying a machine

Buying your own server

- Thousands of riyals up front
- Wait **days** for hardware to arrive
- You fix it when it breaks
- One machine, in one building

Renting from the cloud

- A few halalas per hour, stop anytime
- Ready in about a minute
- The provider handles the hardware
- Data centres **all over the world**

The thing you are really buying is speed of change – being wrong for an hour instead of being wrong for three years.



Think of it as three ways to get a meal

IaaS YOU COOK

The provider hands you raw ingredients – a bare virtual computer. You install everything and you are responsible for it. Most control, most work.

Concretely: **the ECS instance you launch today.**

PaaS HALF-PREPARED

A kitchen with the meal half made; you add your part. No hardware and no operating system to manage.

Concretely: **a managed database, or object storage on Day 4.**

SaaS COOKED AND SERVED

The meal arrives finished. You log in and use it.

Concretely: **Gmail, Netflix, Google Docs.**

Infrastructure = building blocks · Platform = a ready service you build on · Software = the finished product. This week starts at IaaS and climbs toward PaaS.

Six words you will hear every single day

Region WHERE

A geographic area with data centres – Riyadh, Singapore. You pick one near your users.

Availability Zone AZ

An isolated data centre **inside** a region.

ECS THE SERVICE

Elastic Compute Service – Alibaba's virtual machine service. Your computer in the cloud.

Instance THE MACHINE

One running virtual machine. Yours is one instance.

Image THE TEMPLATE

The starting template for an instance, e.g. Ubuntu 22.04. **Careful: "image" means something different for containers tomorrow.**

EIP THE ADDRESS

Elastic IP – a public internet address you attach so the world can reach your instance.

Send a packet and watch exactly where it dies



► Send a packet

EIP attached

Security group: allow 22

`ssh -i my-key.pem root@47.91.x.x`

Press **Send a packet** – then switch the EIP off and send another one.

Two settings you make today that you cannot redo later

- 1** The SSH key pair. This is your login credential. You download a `.pem` file once and you can never download it again. Lose it and the only fix is a new server. Guard it like a password – because it is one.
- 2** The security group. A firewall wrapped around your instance, deciding which ports the outside world may reach. The defaults are fine today; you will learn exactly what it does on Day 3.

Create the key pair first, before you start the instance wizard – otherwise you have to leave the page half-finished to go and make one.

Label each one IaaS, PaaS or SaaS

Your friend Salem is launching a startup and keeps bragging about all the "cloud" tools he uses – but he is fuzzy on what he actually manages versus what the provider manages. Sort them out for him.

1. A bare Ubuntu virtual machine he installs his code on → _____
2. Gmail, which his whole team uses for email → _____
3. A managed database he just connects to (no OS to patch) → _____
4. Salesforce, which his sales team logs into → _____
5. An ECS instance (like the one you are about to launch) → _____

Stuck? Ask: do I manage the operating system? Yes → probably IaaS. Do I just log in and use finished software? → SaaS. Something in between, a managed service you build on? → PaaS.

Build the foundation everything else this week lands on

- 1** Log in to the Alibaba Cloud console and check the [Region](#) at the top matches your instructor's – everyone uses the same one.
- 2** Create the key pair first. ECS → Network & Security → Key Pairs → Create Key Pair. Download the [.pem](#) and save it somewhere safe.
- 3** Create the instance – ECS → Instances → Create Instance. The exact choices are on the next slide.
- 4** Launch and wait for the state to show [Running](#), then copy the public IP and write it down.

```
mv ~/Downloads/_____.pem ~/my-key.pem    # the name you gave your key
chmod 400 ~/my-key.pem                  # you will understand this exactly, this afternoon
```

Match these settings exactly, or this week stops lining up

Billing PAY-AS-YOU-GO

Cheapest for a short course. Not subscription.

Image MUST MATCH

Ubuntu 22.04 LTS. Every command this week assumes it.

Instance type SMALL

The one your instructor names – 2 vCPU / 4 GB is plenty.

Public IP TICK IT

Network tab → Assign Public IPv4 Address, pay by traffic, 200 Mbps. **This is your EIP** – no separate step after launch.

Logon user ROOT

If asked between root and ecs-user, choose root.

Instance name YOURS

Advanced Settings → name it after yourself, so you can find it in a shared tenant.

You can now launch real, reachable cloud infrastructure

- ☐ Your instance shows Running, it has a public IP you have written down, and your `.pem` key file is saved with `chmod 400` applied.

Ping may time out – many firewalls drop it. That is not a failure. Running + a public IP = success. The real proof is SSH, and that is the first thing you do after the break.

 Kahoot: Day 1 · Session 1 – IaaS/PaaS/SaaS, regions and zones, ECS, EIP.
 Bring to Session 2: your EIP and your `.pem` file.

So far you have only clicked at it. Now you drive it.

Real work on a server happens in a terminal, over a connection called SSH – and almost everything for the rest of the week happens there. This is the session where the cloud stops being a website and becomes a computer you operate.

First time really living in a terminal? That feeling is normal and it wears off today. You type the contents of the dark boxes and press Enter; lines starting with **#** are notes, not commands.

The key pair is one lock split into two halves

Your private key .PEM

The file you downloaded and can never download again. It stays on your laptop. You never send it, paste it, or commit it.

The server's public key ON THE VM

The matching half, already installed on the server when you selected the key pair at launch.

They fit together PROOF

The server proves you hold the private half without you ever transmitting it. No password crosses the wire.

```
ssh -i ~/my-key.pem root@<your-EIP>
```

```
-i ~/my-key.pem    → identify me using this private key file  
root@<your-EIP>    → log in as user root, at your public address
```

First connection asks whether to trust the server – type **yes**. When your prompt becomes `root@iZ...:~#` you are in, and that prompt is the server talking, not your laptop.

Everything hangs off one root, and you are always standing somewhere

pwd WHERE AM I

Prints the directory you are standing in right now.

ls -l WHAT IS HERE

Lists the contents, with permissions, owner, size and date.

cd MOVE

Changes directory. `cd ..` goes up one level.

mkdir MAKE

Creates a directory.

```
$ pwd
/root

$ ls -l
-rw-r--r--  1 root root  220 Jul  5 10:00 notes.txt

$ mkdir day1 && cd day1
$ pwd
/root/day1
```

The tree starts at `/`. Everything else is somewhere below it.

Build the mode and watch SSH refuse your key

-rw-r--r--

what `ls -l` shows you

OWNER

r4

W2

X1

4 + 2 = 6

GROUP

r4

W2

X1

4 = 4

OTHERS

r4

W2

X1

4 = 4

644

what you pass to `chmod`

```
$ ssh -i my-key.pem root@47.91.x.x
→ WARNING: UNPROTECTED PRIVATE KEY FILE! Permissions 0644 are too open.
  This private key will be ignored.

$ ./report.sh
→ bash: ./report.sh: Permission denied
```

- chmod 400
- chmod 600
- chmod 644
- chmod 755
- chmod 777

click any r / w / x square to toggle it

SSH refuses this key. Anyone else on your laptop could read it, so OpenSSH will not use it at all. This is the exact error you get if you skip **chmod 400** – and it is why Session 1 told you to run it.

Three groups of three, and one letter in front

```
-rw-r--r-- 1 root root 220 Jul 5 10:00 notes.txt
```

```
-   rw-   r--   r--
type owner group others
```

chmod 400 4,0,0

Owner read only, nobody else anything.
This is what your SSH key needs – and
SSH refuses a key that is more open
than this.

chmod 755 7,5,5

Owner everything; group and others
may read and run. The normal mode for
a program.

chmod +x SHORTCUT

Just add "executable", leaving the rest
alone. What you use on a script.

That **chmod 400** you ran this morning without knowing why? It means only I can read this, and nobody else can touch it.

Watch seven lines go in and one come out

```
$ cat /etc/passwd | grep root
```

7 lines in

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sshd:x:105:65534:./run/sshd:/usr/sbin/nologin
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
```

grep root

0 lines out

Pipe |

Redirect > >>

grep root

grep bash

grep nologin

| wc -l

change the pattern and watch a different set survive

...

One sends output to a command, the other sends it to a file

```
cat /etc/passwd | grep root    # only lines containing "root"
history | grep ssh            # every ssh command you have run

echo "hello cloud" > hello.txt  # > write (OVERWRITES)
echo "another line" >> hello.txt # >> append (adds to the end)
sort < names.txt               # < feed a file in as input
```

The pipe |

Plumbing. Data flows left → right, each command reshaping it. The command on the left never knows the one on the right exists.

> clobbers, >> appends. Mixing these up overwrites a file you meant to add to – the classic first-week mistake.

A value your shell passes to every program it starts

```
export NAME="Cloud Explorer"  # set one
echo "Hello, $NAME"           # read it – $
pulls the value out
echo $HOME                    # a built-in one
env                            # list all of them
```

Why you will care tomorrow

The app you deploy this week reads its own web address out of a variable called `SITE_ADDRESS`. Configuration lives in the environment, not in the code – that is how the same image runs on your laptop and on your server.

A script is just the commands you already type, saved in a file

```
#!/bin/bash
# hello.sh
NAME="$1"           # $1 = the first
argument
echo "Hello, ${NAME:-world}!" # or "world" if
none given
echo "This server is: $(hostname)"

$ chmod +x hello.sh
$ ./hello.sh Salem
Hello, Salem!
```

#!/bin/bash SHEBANG

The first line says which interpreter runs this file.

\$1 ARGUMENT

The first thing you passed in – like a function parameter.

./ RIGHT HERE

"The file in this folder." Without it the shell searches its PATH instead and does not find you.

Fill the blanks so report.sh writes a tidy report

```
#!/bin/bash
# report.sh - usage: ./report.sh "<your name>"

export OWNER="____" # read the first argument into OWNER

echo "=== Server report ===" ____ report.txt # start the file (overwrite)
echo "Owner: $OWNER" ____ report.txt # append
echo "Host: $(hostname)" >> report.txt
echo "When: $( ____ )" >> report.txt # today's date and time
echo "Uptime: $(uptime -p)" >> report.txt

echo "Report saved for $OWNER!"
cat report.txt | grep ____ # pull just the Owner line back out
```

Hints: start the file with a single >, then add every later line with >>. \$(command) runs a command and drops its output straight into your text. Create it with **nano report.sh**; save with **Ctrl+O, Enter, Ctrl+X**.

Log in, prove where you are, and run your script there

- 1 Connect from your laptop terminal.
- 2 Prove where you are – the four commands opposite.
- 3 Make a working area you will reuse tomorrow.
- 4 Complete the task – write and run **report.sh** right there on the server, not on your laptop.

```
ssh -i ~/my-key.pem root@<your-EIP>

pwd
hostname
whoami
cat /etc/os-release | grep PRETTY_NAME

mkdir day1 && cd day1
```

Stuck? `man chmod` and `man grep` (press `q` to quit). Or ask a teaching assistant during lab hours. 😊

You can now control a cloud server from the terminal

- ☐ You connected over SSH and can move around confidently, and `report.sh` runs – building `report.txt` with `>` then `>>`, injecting `$(date)`, and pulling the owner line back out with a pipe.

This morning

Launched real, reachable infrastructure – a running server with a public address.

This afternoon

Connected over SSH, navigated and managed files, set permissions, chained commands, and wrote a script.

Tomorrow

You put your app inside a container and deploy it onto the machine you just built.

 Kahoot: Day 1 · Session 2 – SSH, navigation, permissions, pipes and redirection, environment variables.

IN PARTNERSHIP WITH

